

EEET2588

Real-time Systems Engineering & Design

- Lecture 5:**
- Structured Analysis and Structured Design
 - Unified Modeling Language (UML) Diagrams

Dr Linh Tran

Course note acknowledgment: *Original course notes developed by Roy Ferguson and Samuel Ippolito from RMIT Melbourne*

Lecture 4 Review

- Data Protection and Synchronization Primitives
 - **Mutex**
 - acquiring the mutex within a time limit
 - **Barriers**
 - **Read / Write Locks**
 - **Sleepon Locks**
 - Recursive sleepon locks
 - **Conditional Variables (condvars)**
 - Relocking mutexs and schedule yielding
 - **Semaphores (Named and unnamed semaphores)**
 - Binary semaphores (mutex operations)
 - Counting semaphores
 - Block / Wakeup Process Synchronization
 - Read / Write Lock implementations
 - Controlling bounded buffers, limitations
 - **Message Passing**
- Sequence Diagram examples
- QNX Periodic Timers (communication channels)



[1]

Lecture 5 Overview

- The need for a Rational Design Process
 - Typical Phases in Design Process
 - Project Life Cycle Methodologies
- Structured Analysis and Structured Design
 - Context Diagram
 - Data Flow Diagrams (DFD)
 - Process control
 - Structure Chart
 - Finite State Machines (FSM) Diagrams
 - Hierarchical FSM Diagrams
- Concurrent Design Approach for Real-time Systems (CODARTS)
 - Sequence Diagrams
- Introduction to UML
- Concurrent Object Modeling and Architectural Design (COMET)
- Context Diagrams
- UML Use Cases
 - Diagrams and Written Specs
- Object (or Task Architecture) Diagrams
- State Machine Diagrams (State charts)
- Sequence Diagrams
- Communication (or Collaboration) Diagram

Rational Design Process

Typical Phases in Design Process

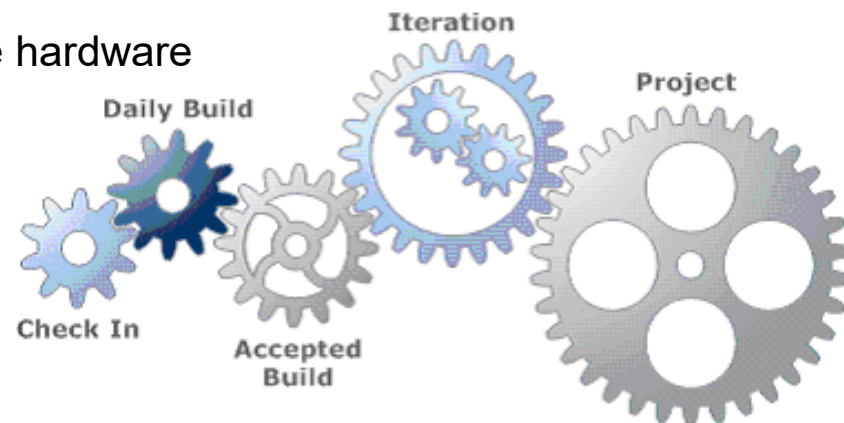
- Preliminary activities for any software development usually involve:
 - Research, analysis, self-education and most importantly, **requirements extraction!!**
- System Functional Design
 - The domain of “**Systems Engineering**” and most likely other professionals
 - E.g. Chemical engineers, process engineers, software and hardware engineers, etc.
 - Identify software and hardware components
 - Clearly define software/hardware interfaces
 - Identify the high-level use case scenarios (user interactions and system behaviour)
- Identification of software/hardware components providers (shopping list)
 - Multiple companies/providers might be involved
- Large projects will require formalized documentation for both the hardware and software – Important for resolving complaints, billing, explanation of system behaviour
 - Software/Hardware Requirements Analysis and Specification
 - Software/Hardware Functional Design
 - Software/Hardware Detailed Design
 - System tests (usually to guide acceptance prior to handover)
- Appropriate design methods are essential for writing and maintaining software.
 - Applies not only RT systems

Black box



Project Life Cycle Methodologies

- After the Design process is finished the Project life cycle commences
 - **Waterfall:** Sequential development with clearly defined deliverables for every phase
 - **Incremental:** Starting from basic partial system features and gradually adding more features until the entire system is completed
 - **Iterative or Adaptive:** Allows flexibility in accommodating new requirements/changes
 - **Agile:** For software applications that need fast-paced evolution
 - Other: Test Driven Development, Rational Unified Process (RUP), etc.
- Selection is based on many factors (project size, delivery speed, end users, location of development teams/resources, stability of the requirements, etc.)
- Regardless of the life cycle mode utilized there are many follow-up stages after the initial design and implementation phases.
 - Integration testing
 - Making sure the software works well with the hardware
 - Software/hardware testing
 - **Software/hardware release**
 - Release Documentation
 - Manual creation, etc.
 - Maintenance phase
 - Long term support



Microsoft MSF shows the various components of an agile life cycle [3]

Structured Analysis and Design (SA/SD)

Real-Time Systems Design Approaches

- **A set of Rational Design Process tool can be used to help:**
- **Structured Analysis and Design (SA/SD)**
 - Devised in the 1970 and 1980 for structured languages like Fortran, Basic, COBOL, Pascal and C
 - Structured Analysis
 - Use Data Flow Diagrams (DFDs)
 - Structured Design
 - Use Structure Charts
(eg developed using Transform and Transaction Analysis)
- **Object-Oriented Design (OOD)**
 - Unified Modeling Language (UML)
 - Now become the de facto standard
 - However (as of 2013), maybe falling out of flavour in industry in preference for informal use. Still used by academics, etc. and is the ideal practice...
 - Can be adapted for RT systems
 - **14+ UML** diagram types
 - We are only looking at ~**5** of them

Overview of an Analysis/Design Process

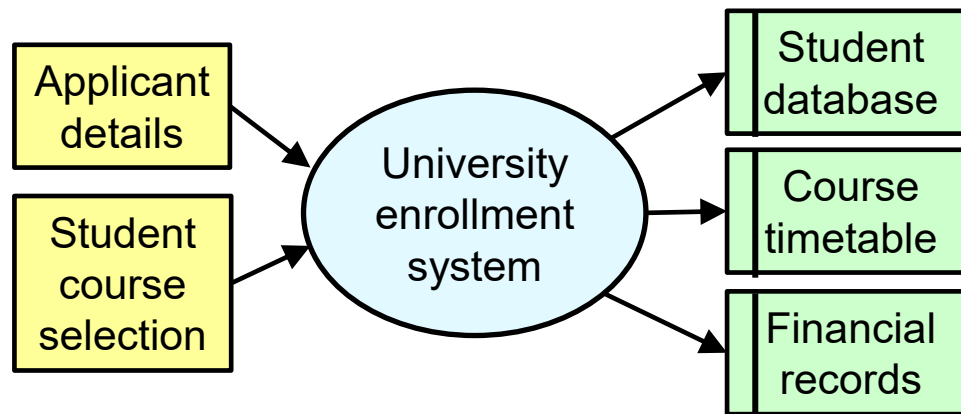
1. Initial Requirements – **Requirements Extraction**
2. Context Diagram (optional, however it is highly suggested that it is used)
 - To identify external entities and interactions
3. Develop Use Cases (scenarios)
4. Develop Sequence and/or Collaboration/Communication diagrams (if useful)
5. Object Discovery
 - All the implementation type diagrams including Task diagrams (Object and Activity Diagrams)
 - Discover Candidate Classes for inheritance, etc. (OOP/OOD only)
6. Further Analysis of Object Behaviours
 - e.g. develop state charts, State Machines Diagrams, etc.
 - Group like tasks/threads
 - Determine final messaging sequences
7. Top-Level Architectural Design
 - Identify concurrency (Tasks and threads)
 - Match to hardware
8. Iterate through again ...

Structured Analysis - Context Diagram

- Also known as a **Context-Level Data-Flow Diagram** or a **Level-0 Data Flow Diagram**
 - Is a high level Data Flow diagram that abstracts the inner details
 - Very similar to the unofficial use case diagrams used with UML design.
- **Context Diagram** shows the system as a single high-level process
 - Represent all external entities that may interact with a system
 - Diagram pictures the system at the center, with no details of its interior structure, surrounded by all its interacting systems, environments and activities
 - **System Entity**: The abstracted single high-level entity
 - **Actors Entities**: Labelled boxes or labels around the single system entity
 - **Relationships**: Labelled lines between the entities and system
- Notation is flexible, as many changes have occurred over the years.
Generally uses are:
 - System Process represented with a **Circle** around the name of the process
 - External Entities represented within a **Rectangle**
 - Data Stores (two horizontal, parallel lines or sometimes an ellipse)
 - Data Flows (curved or straight line with arrowhead indicating flow direction)

Structured Analysis - Context Diagram

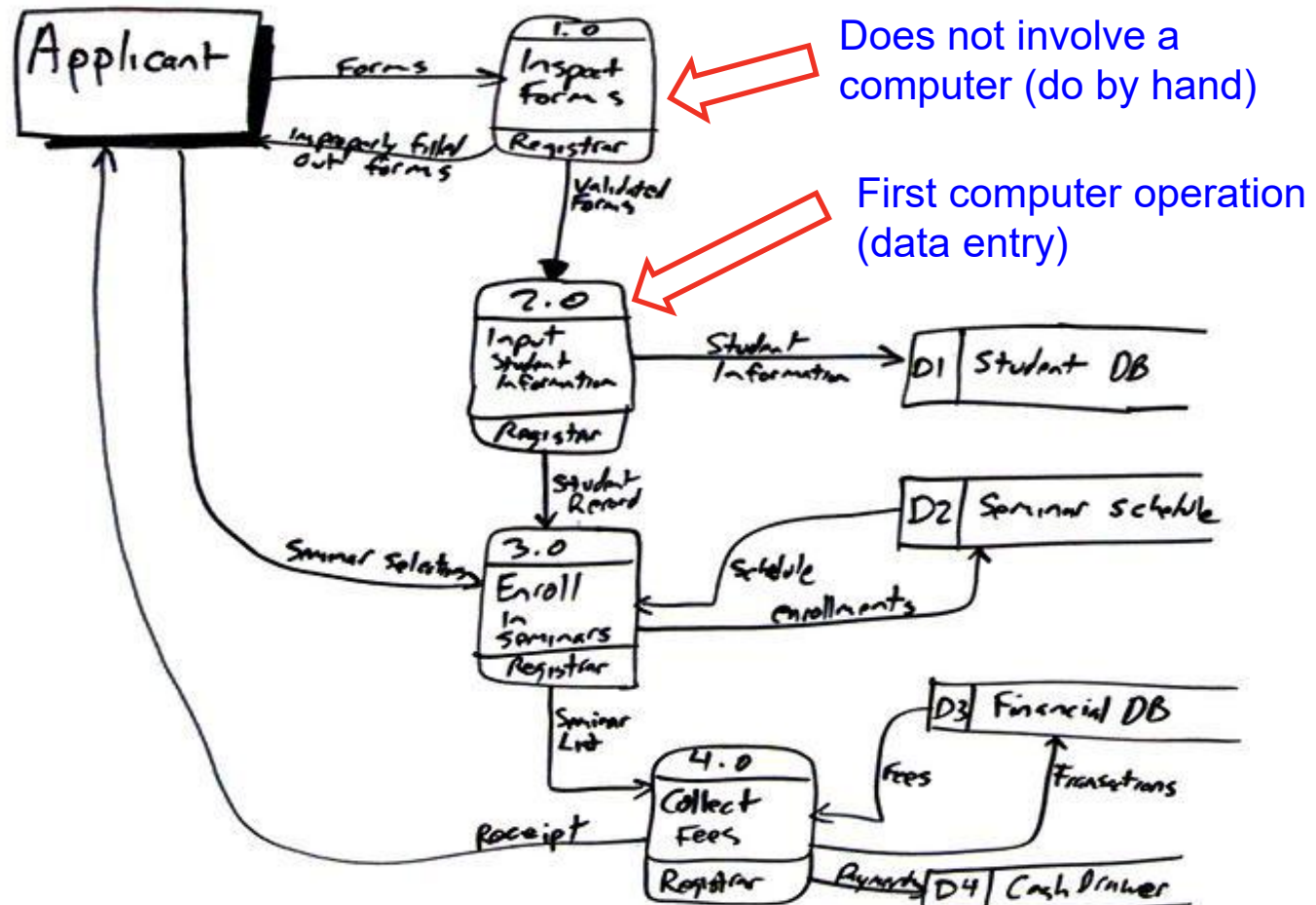
- Some of the benefits of a Context Diagram are:
 - Shows the scope and boundaries of a system at a glance including the other systems that interface with it
 - No technical knowledge is assumed or required to understand the diagram
 - Easy to draw and amend due to its limited notation
 - Easy to expand by adding different levels of DFDs
 - Can benefit a wide audience including stakeholders, business analyst, data analysts, developers
- A context Diagram provides no information about the timing, sequencing, or synchronization of processes



Context diagram for a University Enrollment system

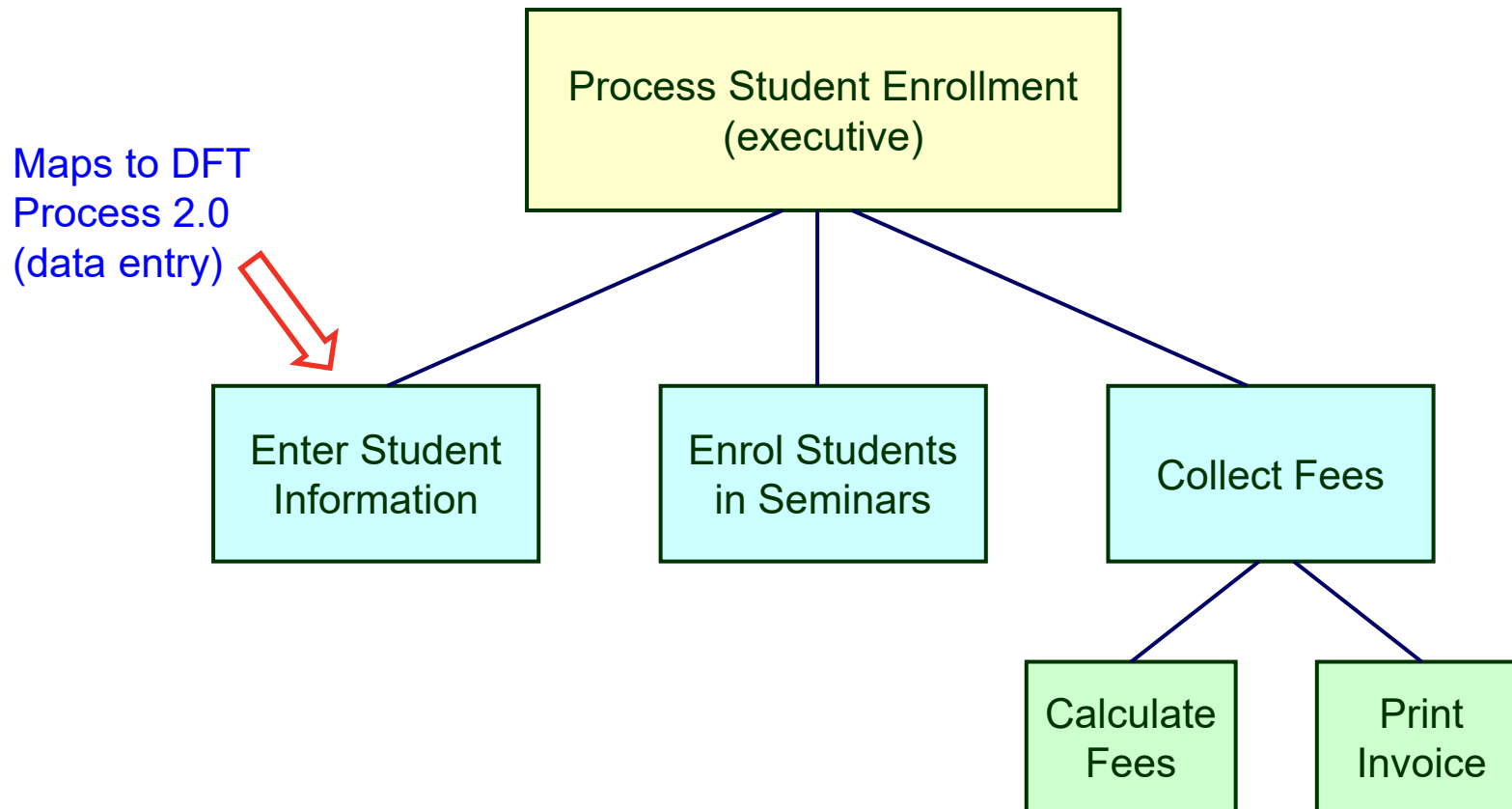
Structured Analysis - Data Flow Diagrams (DFD)

- Second level of detail/decomposition is labeled a “Level-1 DFD”
- Initial design starts with a hand drawn diagram – not perfect, will need to be altered
 - Breaks down the problem in process and data flowing between the processes
 - Process are numbered to provide detail in the order of execution
- Level-1 DFD should capture everything – not only the software aspect of the system

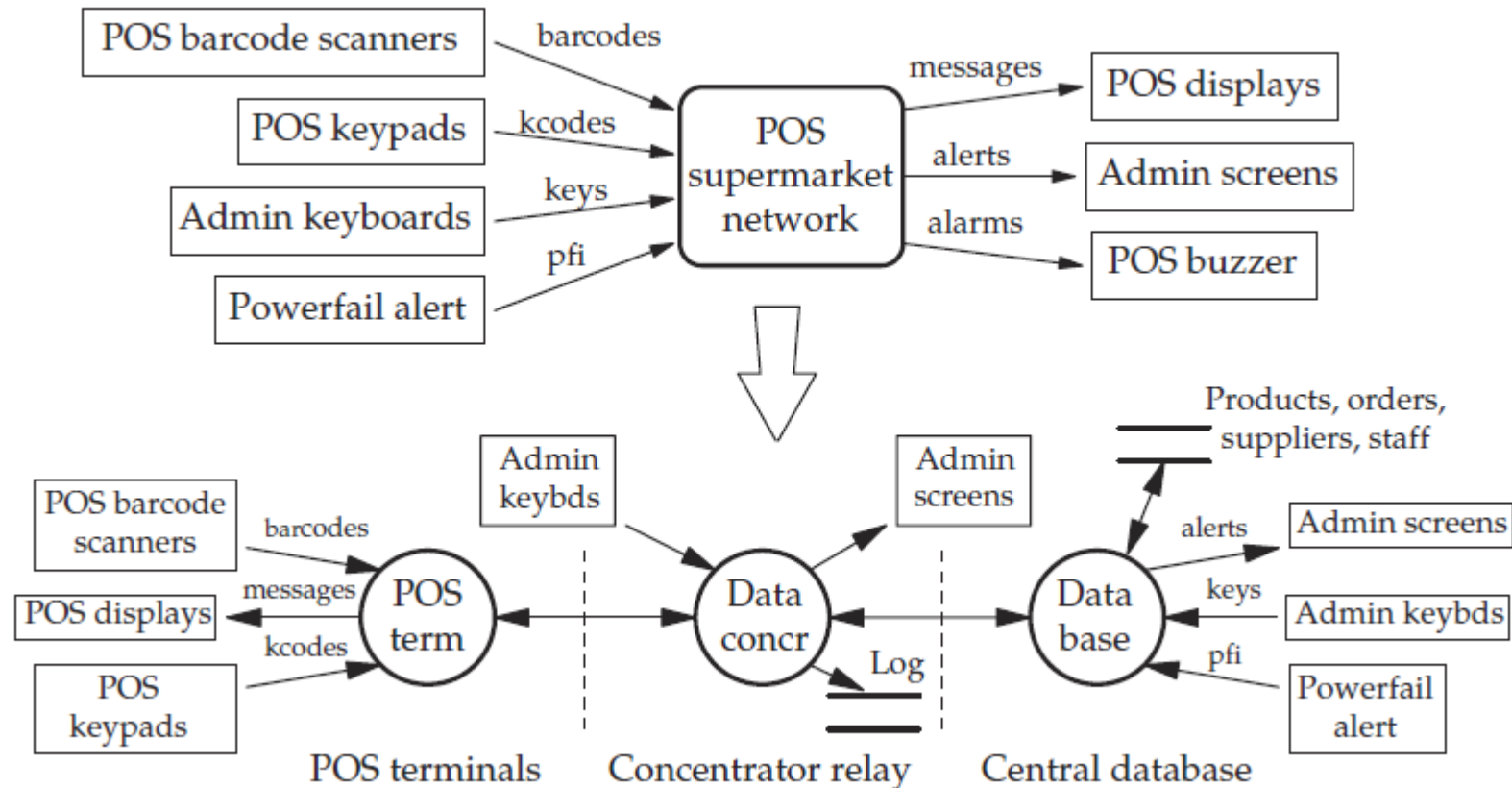


Structured Design - Structure Chart

- To define the software aspects, the DFD can then be turned into a structure diagram
 - Flow sequence of the master process (executive) calls other processes or functions
 - Function called in the priority they are drawn
 - Usually: **Top to Bottom**, **Left to Right**
- There are formal approaches to get from a complex DFD to a formal structure chart



Context Diagram (Level-0 DFD) to a Level-1 DFD

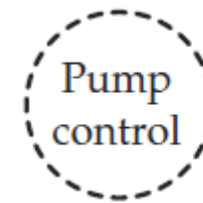
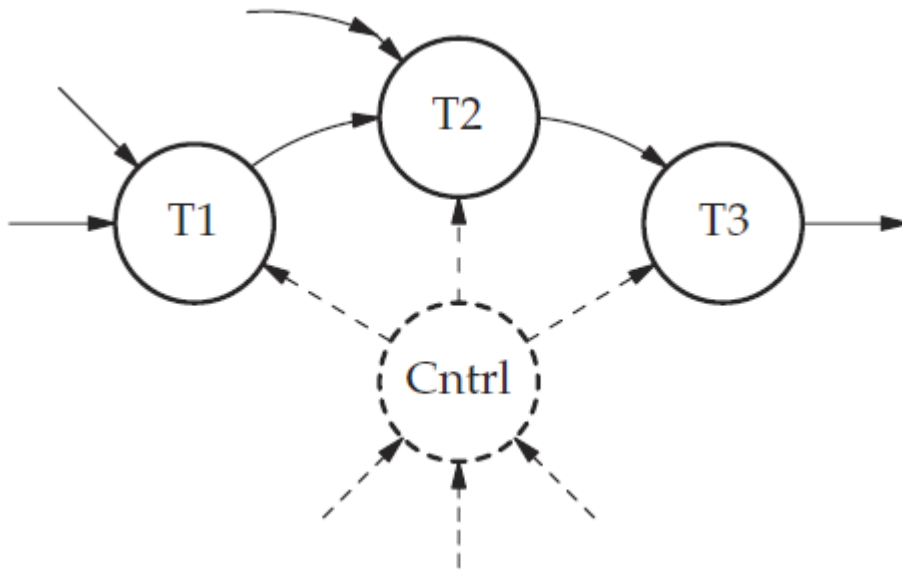


- Maintaining consistency between the context diagram and the derived data flow diagrams is essential.
- A serious deficiency of the DFD schematic are their failure to fully specify the *timing* and *functionality* of each component data transformation.

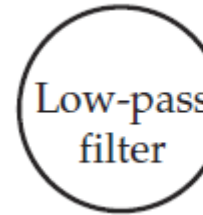
[Williams, *Real-Time Systems Development*, Ch.11, page 259]

DFD with Control Process

- DFD diagrams were updated in 1985 to make them more suitable to for use with real-time systems.
 - ‘Control Processes’ entities were added, which specifically act on the data processes as schedulers.
 - In this way the scheduling activity could be explicitly separated from activity on the incoming data stream.
 - Other notations were updated to better separate trigger events from messages, which can take the form of parameter passing, FIFO queues or global data value changes.



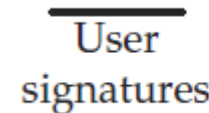
Control process



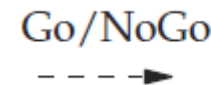
Data process



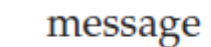
Data source/sink



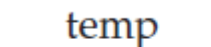
Data store



Event flow



Discrete data flow



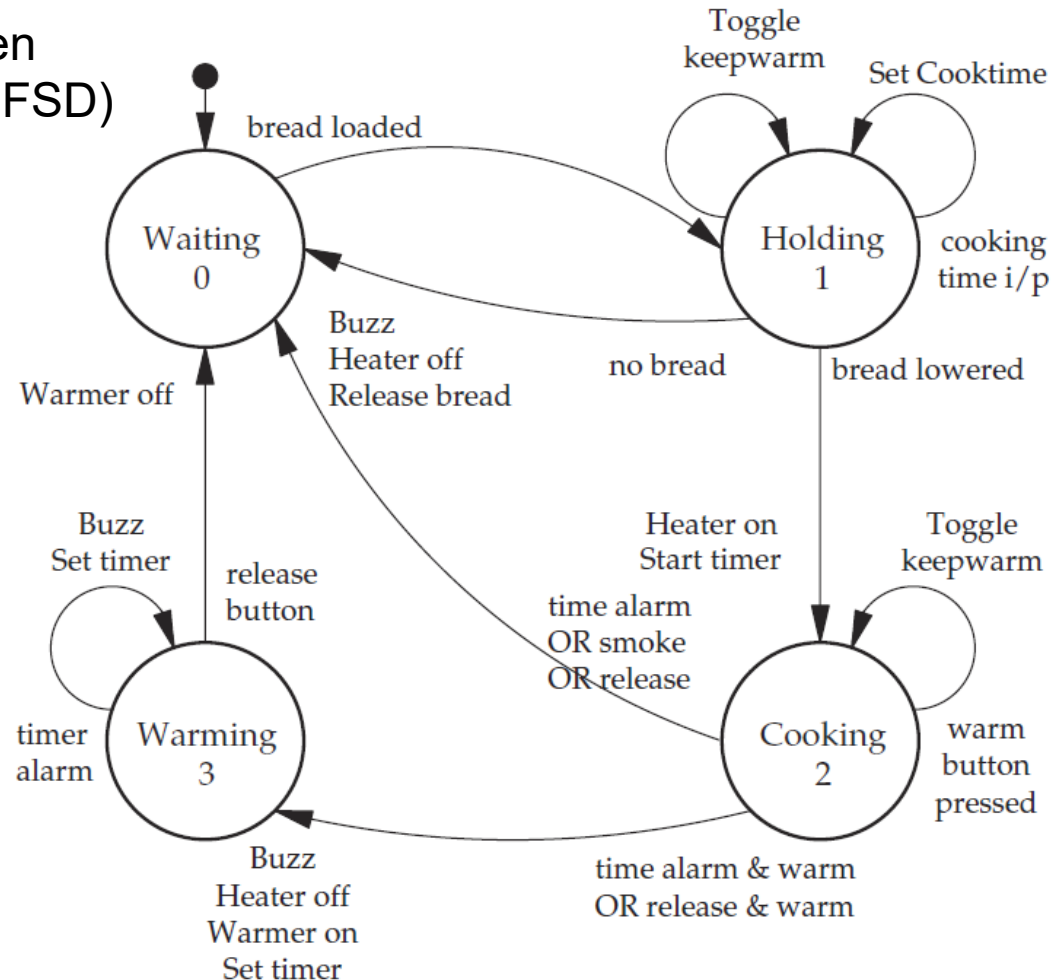
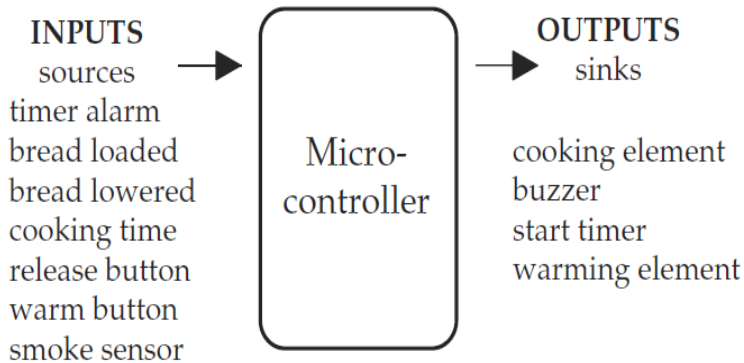
Continuous data flow

[Williams, *Real-Time Systems Development*, Ch.11, page 270]

Finite State Machines

- The use of Finite State Machines (FSM) diagrams and coding methodologies is utilized by several software design methods.
 - Depending on the problem, FSM can provide strong support during both design and implementation phases of software development.
- Start with a context diagram and then progress to a Finite State Diagram (FSD)

Context diagram for a (complex) Toaster

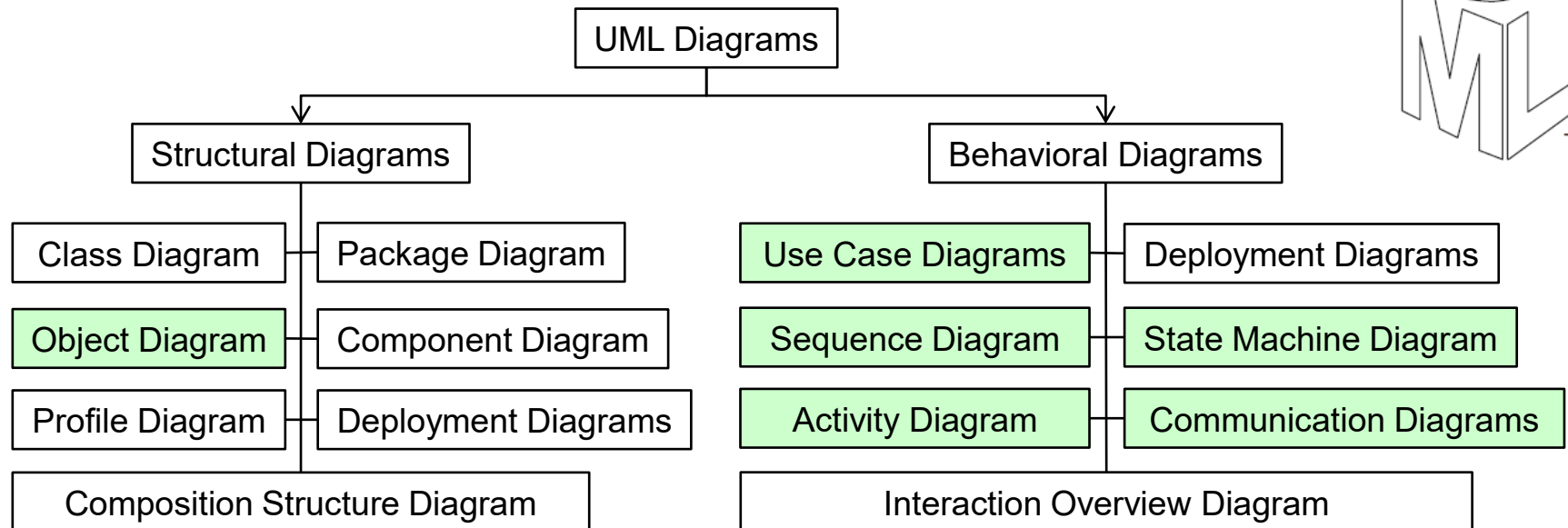


[Williams, *Real-Time Systems Development*, Ch. 5, page 107-109]

Unified Modeling Language (UML)

Unified Modeling Language (UML)

- UML is a collection of graphical drawings used to describe systems by expressing their constructs and relationships.
 - UML was developed well after SA/SD was well established (mid 1990's)
 - UML 2.0 is more complex and extensive in nature (post 2000's)
 - UML has taken over SA/SD approaches and has now become the de facto standard
- Many UML diagrams share a commonality with SA/SD diagrams
 - UML has been adapted for real time design



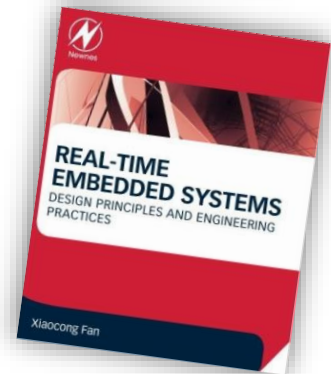
- **Static view**: captures the *static structure* of the system using objects, attributes, operations and relationships.
- **Dynamic view**, which captures the *dynamic behavior* of the system by visualizing collaborations among objects and internal changes of object states.

Unified Modeling Language (UML)

- UML was developed by Grady Booch, Ivar Jacobson & James Rumbaugh in 1994/95
 - Booch, Rumbaugh and Jacobson, *The Unified Modeling Language User Guide*
 - Rumbaugh, Jacobson and Booch, *The Unified Modeling Language Reference Manual*
 - Jacobson, Booch and Rumbaugh, *The Unified Software Development Process*
- Fowler, Martin and Kendal Scott, *UML Distilled: A Brief Guide to the Standard Object Modeling Language*, Addison-Wesley
- In 2005 a major update of UML was released: UML 2.0
 - Improve the language further to reflect new experience on usage of its features.
 - Several updates were then released to keep it relevant.
 - In 2012 UML 2.5 was last official release.
- Over time UML has been treated as a design silver bullet
 - This has led to problems in its usage.
 - Misuse of it includes excessive usage of it (design every little part of the system's code with it, which is unnecessary)
- Some useful web sites:
 - UML 2 Specifications: www.uml.org
 - UML 2.5 Diagrams Overview: www.uml-diagrams.org/uml-25-diagrams.html
 - Helpful UML tutorial: www.tutorialspoint.com/uml/index.htm
 - UML Diagram Creator (scripting): www.plantuml.com (Editor: www.planttext.com)

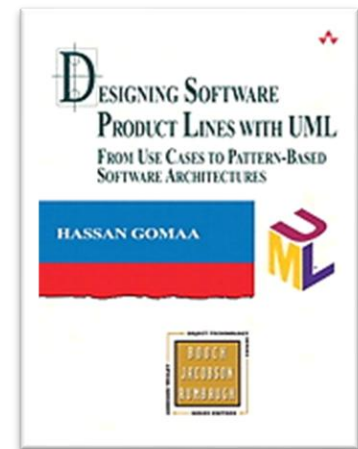
Drawing Packages and References

- Microsoft VISIO
 - Visio Stencil and Template for UML 2.5
 - <http://softwarestencils.com/uml/index.html>
- Modelio open source modeling environment
 - https://www.modelio.org/modelio-download-archive/cat_view/925-modelio-tool/924-modelio-340.html
- UML Diagram Creator (scripting language)
 - Java based engine: www.plantuml.com
 - Useful Editor: www.planttext.com
- Useful Books:
 - UML books by Booch, Rumbaugh, Jacobson
 - UML books by Gomaa, Fowler, Martin and Kendal Scott
 - Williams, Real-Time Systems Development, 2006
 - Xiaocong Fan, *Real-Time Embedded Systems Design Principles and Engineering Practices*, 2015
- Useful web resource: www.uml-diagrams.org/uml-25-diagrams.html
- SysML (Systems Modeling Language) Open Source Project: www.sysml.org



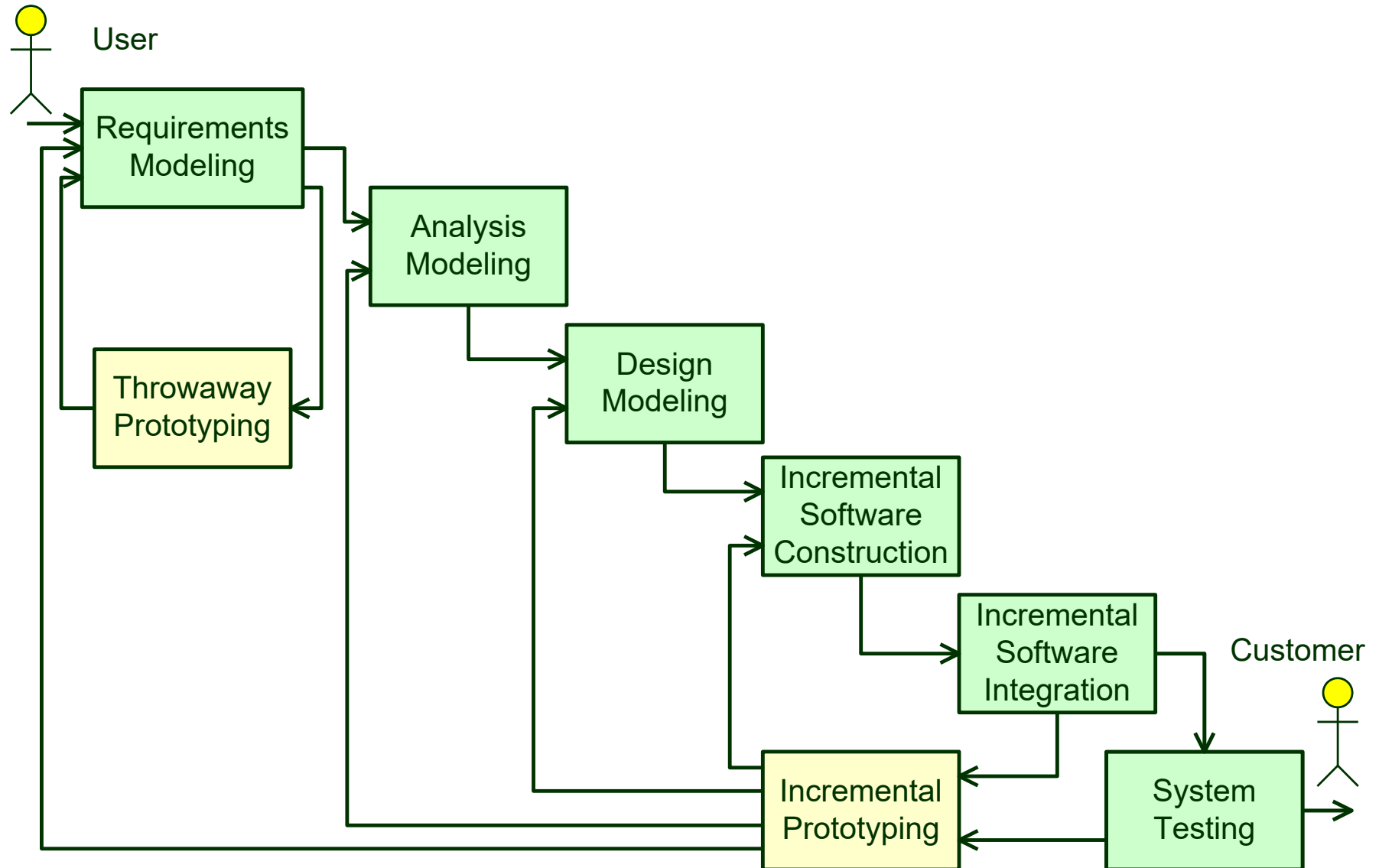
Concurrent Object Modeling and Architectural Design (COMET)

- Gomaa developed a Software design Methodology especially designed for UML design called Concurrent Object Modeling and Architectural Design (COMET).
- COMET is a highly iterative object-oriented software development method
 - The functional requirements of the system are defined in terms of actors and use cases.
 - Each use case defines a sequence of interactions between one or more actors and the system.
 - A use case can be viewed at various levels of detail.
- It addresses the requirements, analysis, and design modeling phases:
 1. **Requirements Modelling**
 - Gather Requirements
 - Document Requirements
 - Develop initial (Context diagram and) Use Cases
 2. **Analysis Modelling**
 - Refine and develop specifications
 - Develop final Use Cases
 3. **Design Modelling**
 - Architectural Modelling
 - Involves collaborations of packages, tasks or processors



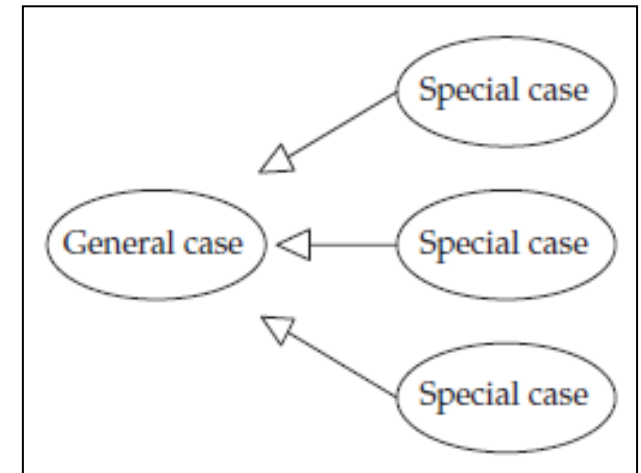
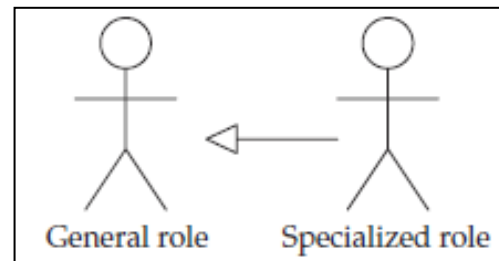
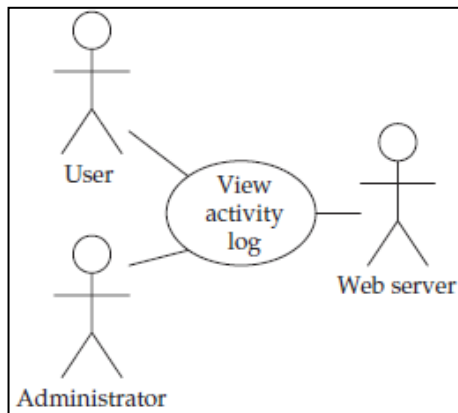
[4]

COMET Life Cycle (Gomaa)



UML Use Cases Diagrams

- The next step of the is to develop the specific Use Case diagrams for each scenario
 - Break the context diagram down into more detail
 - However do not consider data types or transfer rates (similar to context diagrams).
- Use-Case diagrams are a set of scenarios tied together by a common user goal
 - Scenarios should be described in words and/or appropriate diagrams
 - (state machines or sequence diagrams can be used for complex descriptions)
- Primary Scenarios (i.e. how the software should work under normal situations)
 - Elaborate the major ways the Use Cases are realised in practice
- Secondary Scenarios (i.e. how should the software deal with potential erroneous input or situations)
 - Variants to explore the less common cases
 - Exception handling
 - Safety issues



[Williams, *Real-Time Systems Development*, Ch. 13, page 286]

Use Case Diagram (Example: Elevator System)

- Actors

- Can be human or an external system
- Primary actors (often human) initiate a use case scenario
- Secondary actors (often external systems) participate in the use-case by receiving outputs and providing inputs

Diagram 1

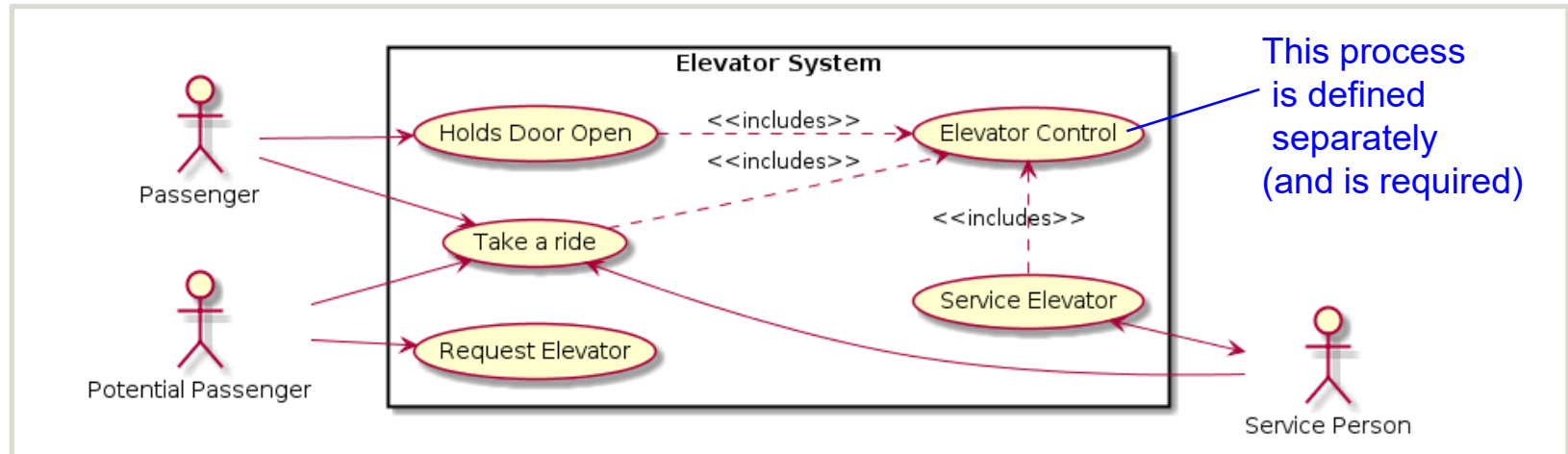
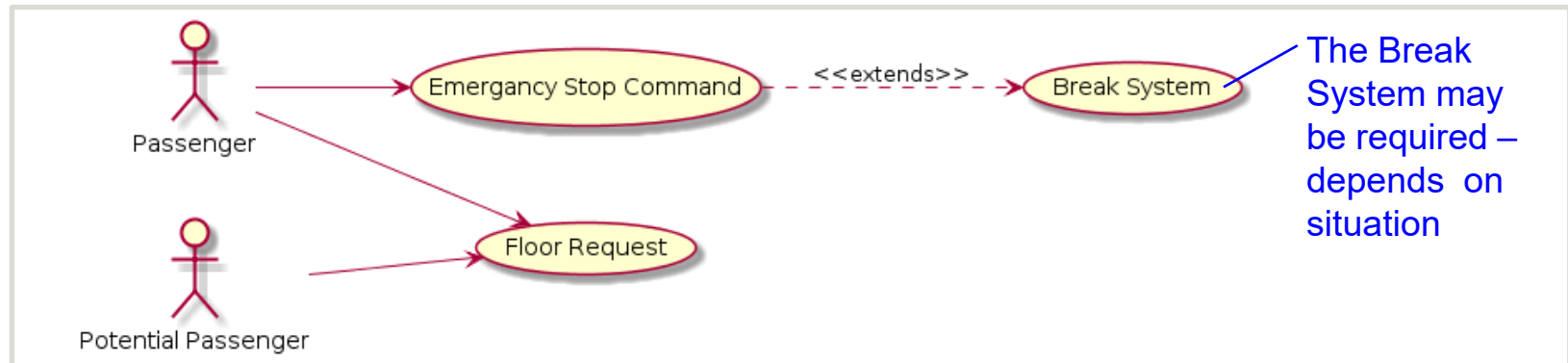


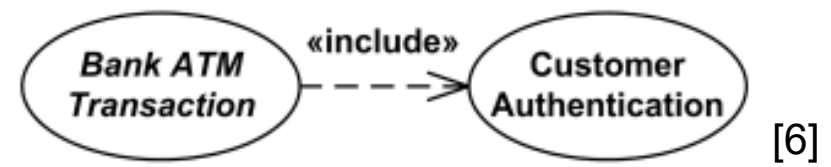
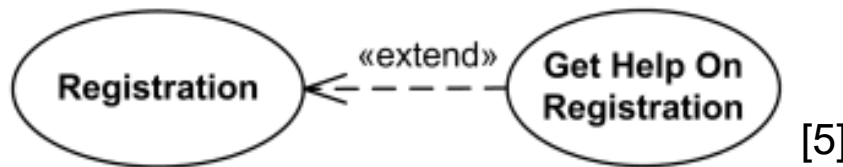
Diagram 1.1



Use Cases generated using: www.planttext.com

UML Use Cases Diagrams

- **<<extend>>** is a directed relationship that specifies how and when the behavior defined – Usually optional, as “Registration” is complete and meaningful on its own.
- **<<include>>** is a directed relationship between two use cases which is used to show that behavior of the included use case (the addition) is inserted into the behavior of the including (the base) use case.



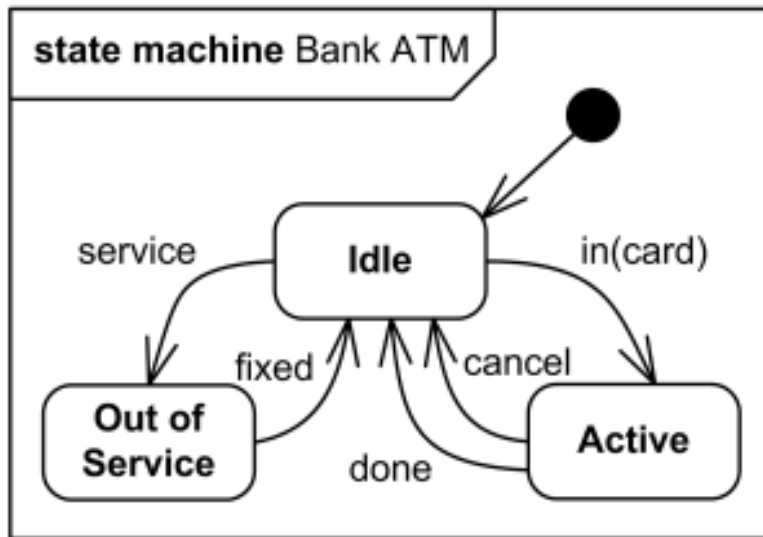
Design Phases	How use cases are used
Analysis	- Suggest large scale partitioning of the domain - Provide structuring of analysis objects - Clarify system and object responsibilities - Capture and clarify new features as they are added during development - Validate analysis model
Design	Validate the elaboration of analysis models in the presence of design objects
Coding	- Clarify purpose and role of classes for coders - Focus coding efforts
Testing	Provide primary and secondary test scenarios for system validation
Deployment	Suggest iterative prototypes for spiral development

Written Use Case Specifications (Example: Elevator System)

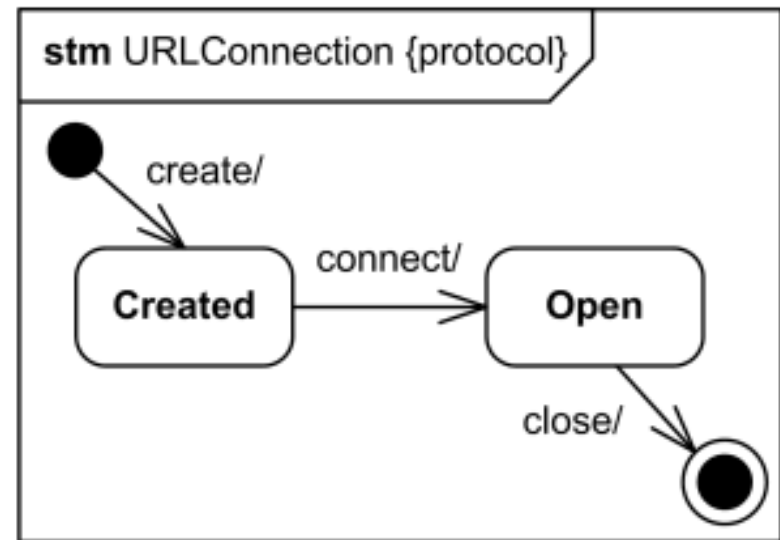
- **Potential Passenger** - “Request Elevator” Use Case
 1. Presses “Up” or “Down” floor button
 2. Waits till an elevator going in the requested direction stops at the floor
 3. Enters elevator
 - becomes a “**Passenger**”
- **Passenger** - “Take a Ride” Use Case
 1. Elevator going in requested direction stops at floor and passenger enters
 2. Passenger presses button for destination floor
 3. If new destination is in same direction elevator is traveling it will be serviced when the elevator reaches that floor
Else if new destination is in opposite direction, it will be serviced only after all requests in current direction are serviced
 4. Elevator moves in current direction and stops at next pending destination or pickup floor
 5. When elevator has serviced all pending destinations and pickup requests in the current direction it will reverse direction
 6. Passenger leaves elevator at destination floor

UML - State Machine Diagrams

- *Dynamic models* can be analysed by creating State Machine Diagrams
- Two types of UML State Machine Diagrams (UML 2.x specifications)
 - Behavioral state machine - used to specify **discrete behaviour**
 - Protocol state machine - used to express a **usage protocol** or a **lifecycle**
- Behavioral state is used to specify discrete behavior of a part of designed system through finite state transitions.



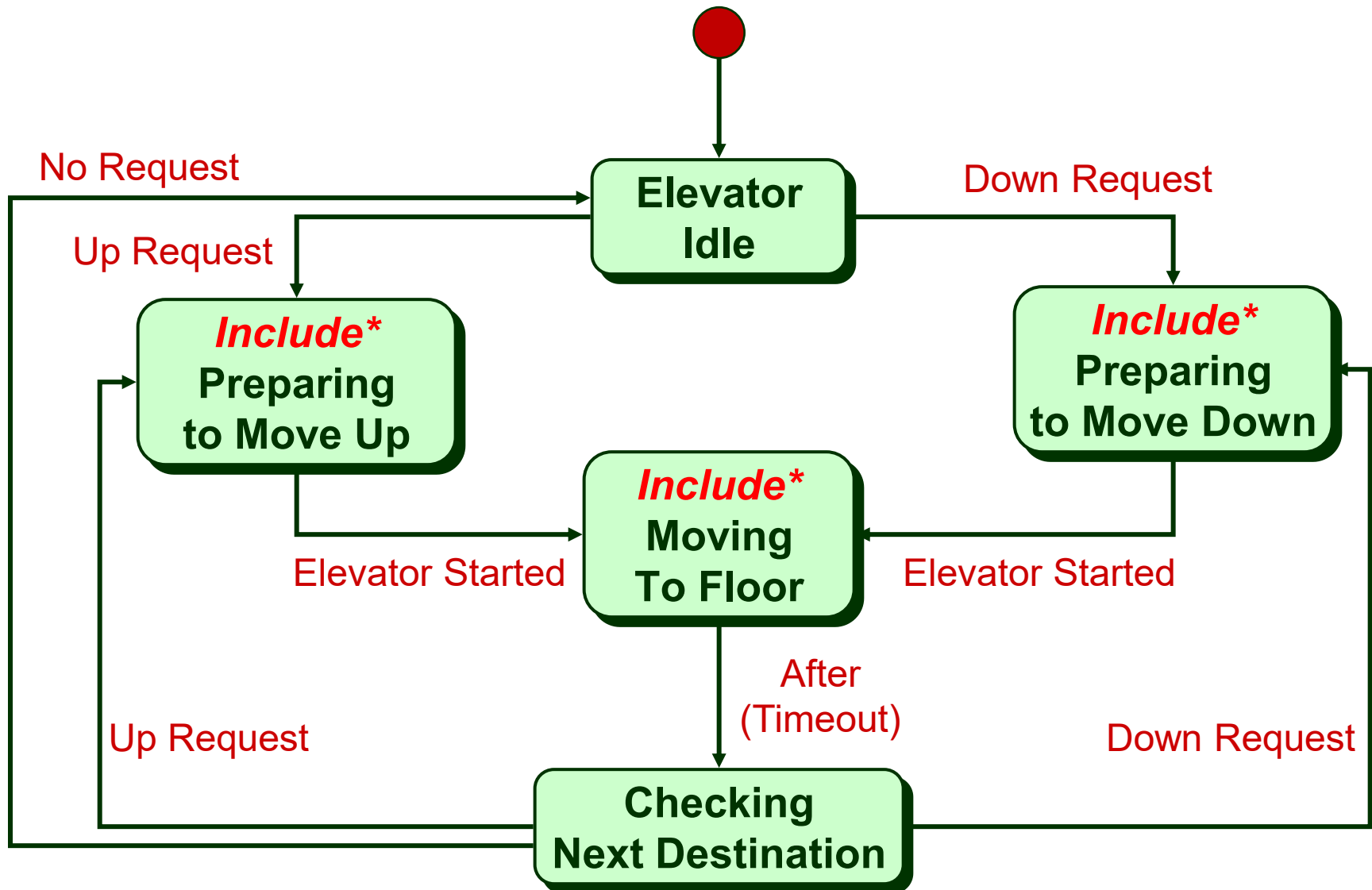
High level behavioral state machine for bank ATM



Protocol state machine for URLConnection class

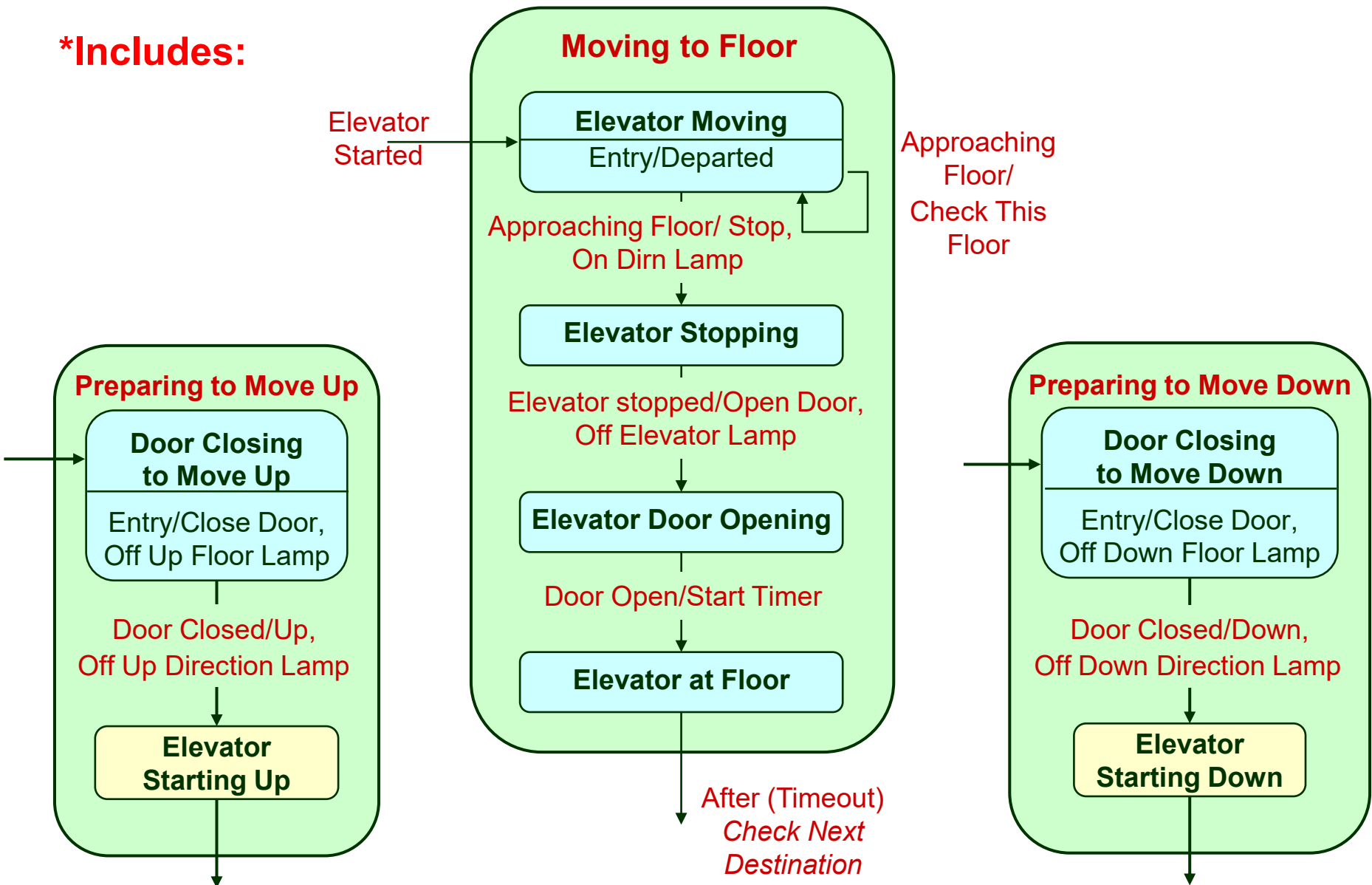
References: www.uml-diagrams.org/state-machine-diagrams.html#behavioral-state-machine
www.uml-diagrams.org/protocol-state-machine-diagrams.html#protocol-state

State Machine Diagrams (Example: Elevator System)



State Machine Diagrams (Example: Elevator System)

***Includes:**



COMET – Design Modeling

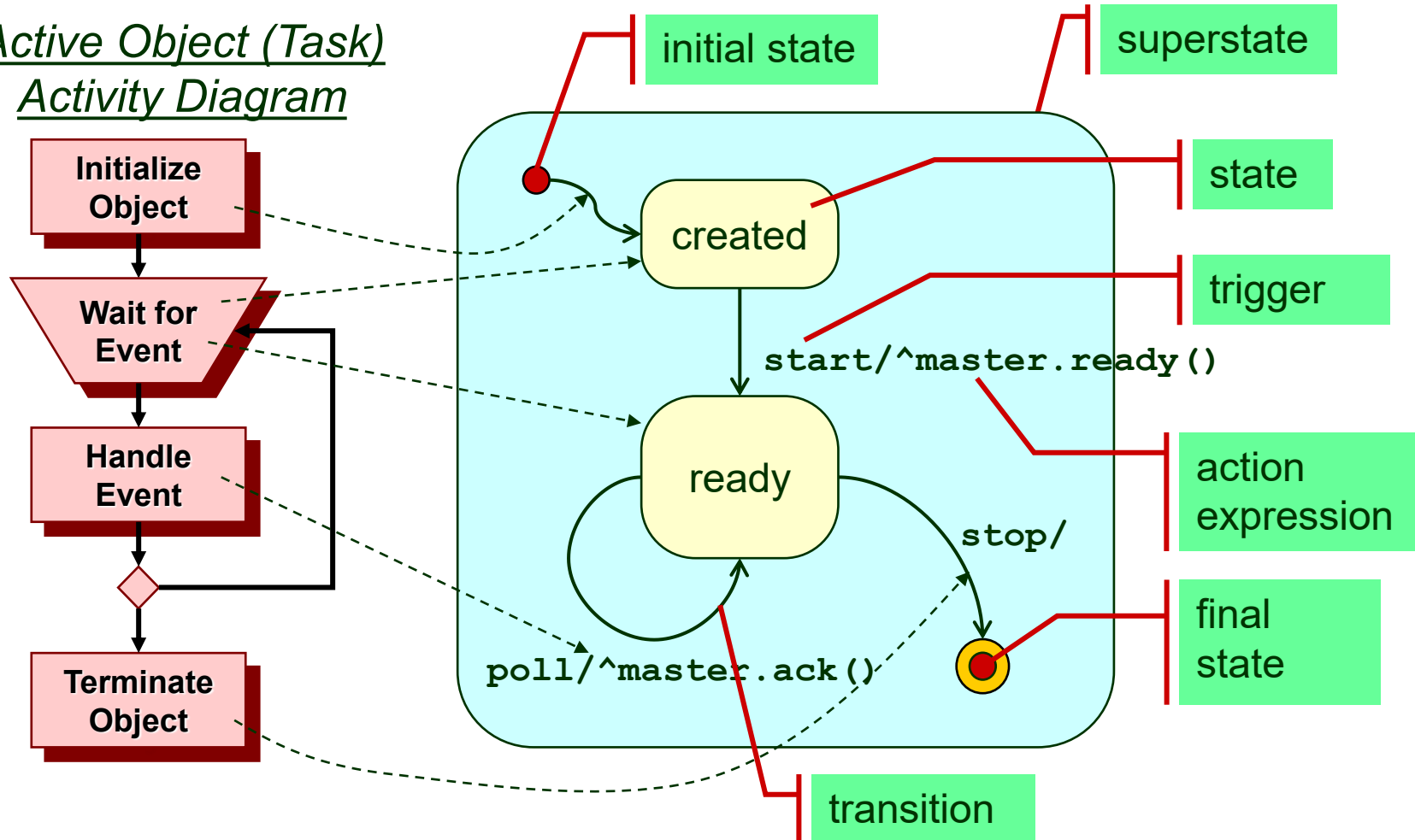
- The Design Modeling phase is used to design the software architecture of the system.
 - The analysis model is mapped to the operational environment
 - Distributed system, single core, multi-core, etc.
 - The analysis model (which emphasizes the problem domain) is mapped to the design model (which emphasizes the solution domain).
 - Special consideration is given to designing distributed subsystems as configurable components that communicate with each other using messages.
- The model describes the whole system as well as its subsystems in detail
 - Use a UML deployment diagram to show processors (if more than one) and major modules or tasks for each processor
 - For each processor use a task/object or activity diagram to show the main tasks (if more than one)
- To tease out specifics you can either replace the traditional object diagram or add:
 - Collaboration diagram (UML 1) or Communication diagram (UML 2)
 - or an Active Object diagram, which is good for showing message flows between tasks
- Could use a CODARTS Data Flow or Task diagram to capture the alignment of objects, task functions and communication channels

UML - State Machine Diagrams (aka: Statechart Diagrams)

- State Machine Charts/Diagrams

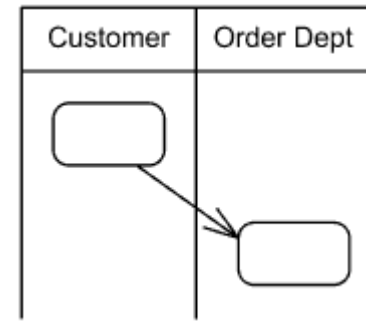
- Are used to look at behaviour of a single object across one or more use cases in detail
- Can be useful for concurrent processes (just like in SA/SD FSM diagrams)
- Often used in Real-Time Systems

Active Object (Task) Activity Diagram



Capturing Behaviour

- For capturing behaviour across many use cases or many threads consider an activity diagram
 - **They capture flow of control or object flow** with emphasis on the sequence and conditions of the flow



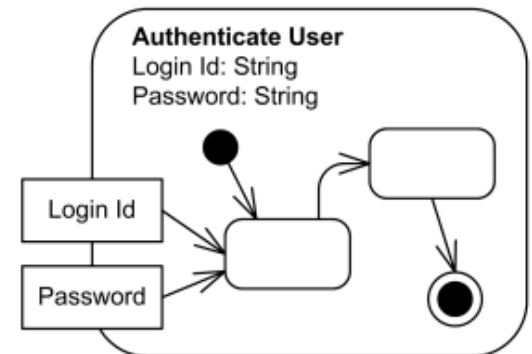
Example of an Partitioned Activity Diagram [7]

- For describing the behaviour of several objects in a single use-case, consider using one of the following **implementation diagrams**:

- Communication diagrams (known as collaboration diagrams in UML 1.x)
- Object Diagrams (or Task diagrams)
- Sequence diagrams,
- Timing diagrams (new in UML 2.x)
- Interaction overview diagrams (new in UML 2.4.1)

- Sequence Diagrams

- Good for a sequence of things that occur in order
- Unique to a specific situation (or sequence)
 - 1 view point per diagram

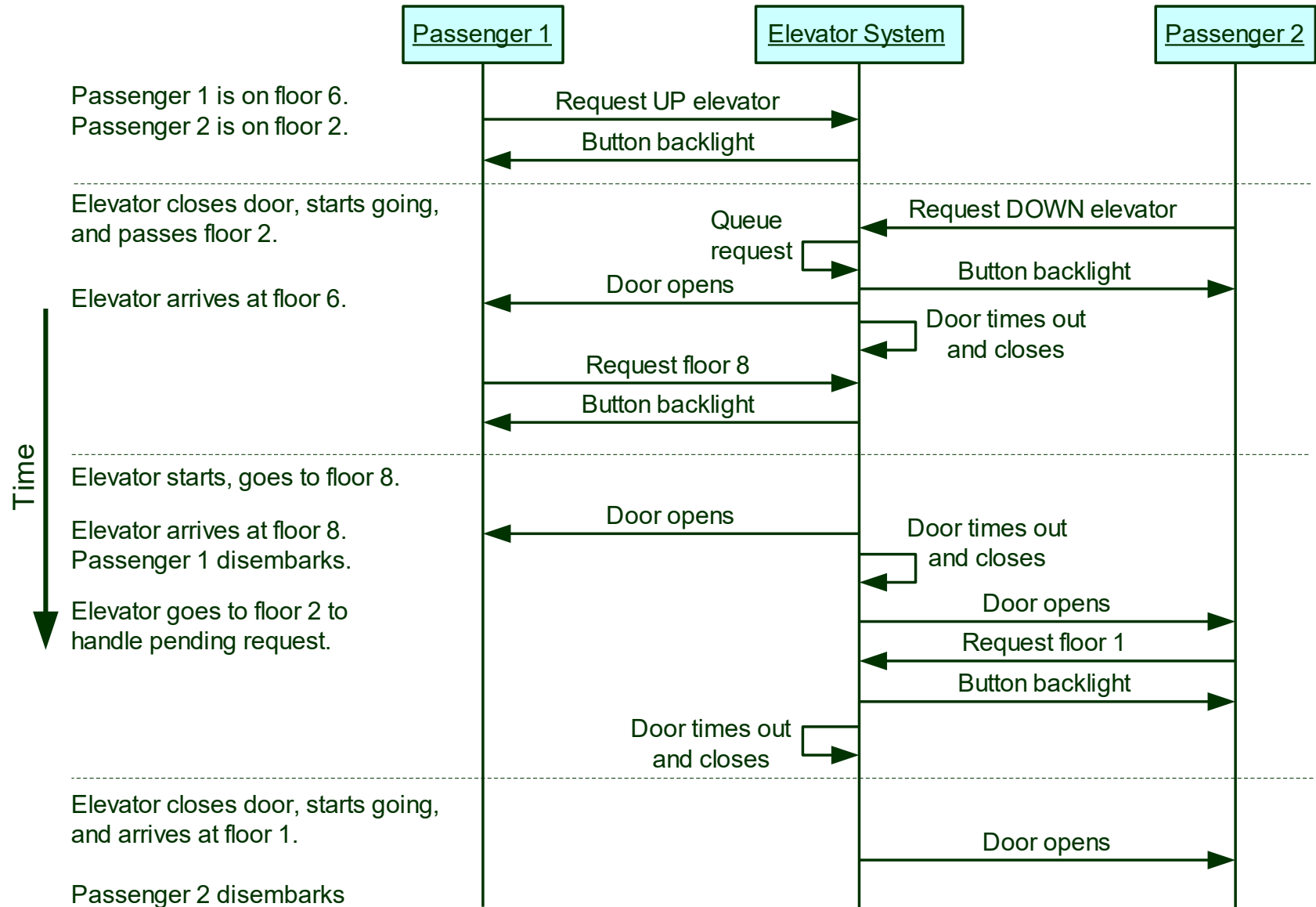


Authenticate User Activity Diagram [7]

- Communication (Collaboration) Diagrams

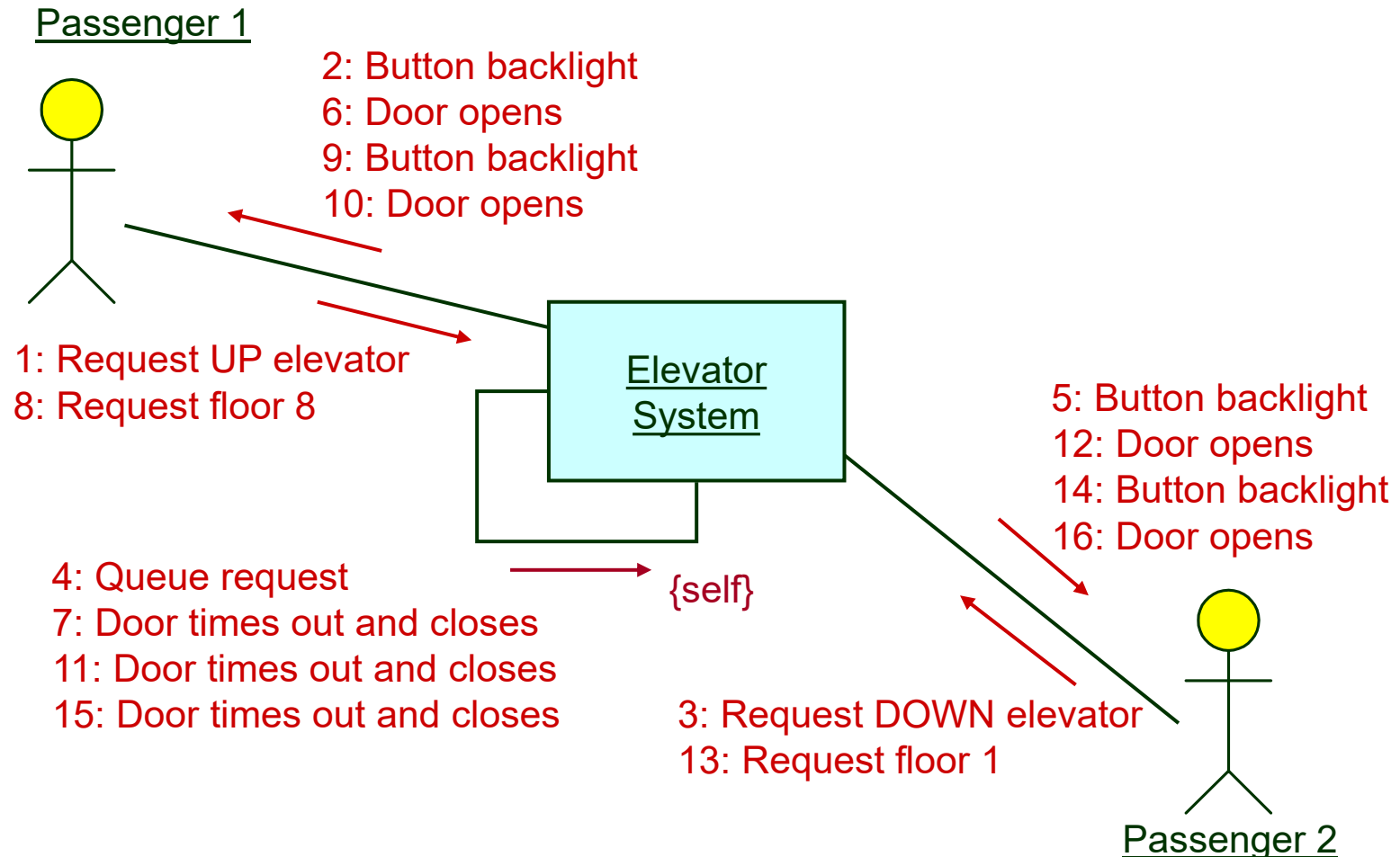
- Can use layout to indicate how objects are statically interconnected and may be used to show concurrent tasks and messages between them

Sequence Diagram (Example: Elevator System)



Communication Diagram (Example: Elevator System)

- Communication diagram (called collaboration diagram in UML 1.x) shows interactions between objects and/or parts (represented as lifelines) using sequenced messages in a free-form arrangement.



Finally, what is your thought?

- How do you think about the tool?
- How do you think other people think about the tool?
- Which diagram can be useful? And when?

References

Images and content used in this presentation can be found:

- [1] <http://www.econlife.com/opportunity-cost-trade-offs-of-municipal-finance/>
- [2] <https://magcos.com/developing/>
- [3] http://www.ittoday.info/Articles/Best_Life_Cycle_for_Your_Project.htm
- [4] <http://www.informit.com/articles/article.aspx?p=349038&seqNum=7>
- [5] <http://www.uml-diagrams.org/use-case-extend.html>
- [6] <http://www.uml-diagrams.org/use-case-include.html>
- [7] <http://www.uml-diagrams.org/activity-diagrams.html>

Sequence Diagrams created using plantUML http://plantuml.com/PlantUML_Language_Reference_Guide.pdf

On the web based editor made available at: <http://www.planttext.com/>

Acknowledgment

I would like to Thank and Acknowledge the support of: Dr. *Samuel Ippolito*, the Course Coordinator of corresponding “EEET2166 /1262 Real-time Systems Engineering & Design” course from RMIT Melbourne.